StockSense AI

Project Requirements & Development Blueprint

Version 1.0 | Full Stack AI Web Application

---

1. Product Overview

StockSense AI is a full-stack, production-deployed web application that acts as an AI-powered stock research assistant. The core problem it solves: users on platforms like Groww can see stock prices but have no guidance on what action to take. StockSense AI fills that gap.

A user searches for any stock, and the app fetches real price history, calculates technical indicators (RSI, MACD, Moving Averages), finds historical patterns in that stock's behavior, and sends all of it to an LLM (Claude or GPT) which responds with a plain-English buy/hold/sell suggestion and reasoning.

The three core pillars of the product:

- AI Advisor — LLM reads real indicator data and gives a reasoned suggestion
- Backtesting Engine — user tests any strategy on 5 years of historical data
- Portfolio Tracker — user tracks holdings with live P&L and AI hold/sell calls

Target user:

Retail investors in India who use Groww or Zerodha but want a research layer that tells them what to do, not just what the price is.

---

2. Tech Stack

Part	Technology	Reason
Frontend	React + Vite	Simple, fast build tooling, no SSR complexity needed
Backend	Python + FastAPI	All data work (indicators, backtesting, ML) lives naturally in Python
Stock data	yfinance (Python library)	Free, no API key, covers NSE/BSE stocks

Indicators	pandas-ta library	RSI + MACD in 2 lines of code, built on pandas
AI	Claude API (claude-sonnet-4-20250514)	Sends stock data context, receives suggestion
Database	PostgreSQL on Supabase	Free tier, managed, includes auth
Auth	Supabase Auth	JWT-based, done in a day, no custom auth needed
Charts	Recharts	React-native charts, works well with financial data
Cron jobs	APScheduler (Python)	Runs daily morning scan inside FastAPI app
Email alerts	Resend.com	Free tier (100/day), simple REST API
Frontend deploy	Vercel	Free tier, auto-deploys from GitHub
Backend deploy	Render	Free tier, supports Python, persistent background workers

---

3. Features (MVP Scope)

3.1 Stock Detail + AI Suggestion

The core feature. User searches any ticker (e.g. TCS.NS, INFY.NS). App fetches 1-year price history via yfinance. Backend calculates:

- RSI (14-period) — identifies overbought/oversold conditions
- MACD + Signal line — identifies momentum and crossovers
- 50-day and 200-day Moving Averages — identifies trend direction
- Volume trend — confirms price movements

Pattern engine then looks back at the stock's history: "The last N times RSI dropped below 35, the stock recovered X% on average over Y days." This context + indicator data is sent to the LLM which returns a plain-English suggestion.

3.2 Backtesting Engine

The star feature. User selects a stock, a strategy (e.g. Buy when RSI < 30, Sell when RSI > 70), and a date range (up to 5 years). The backend simulates all trades that would have occurred and returns:

- Total return (%)
- Number of trades
- Win rate (% of profitable trades)
- Max drawdown
- Equity curve (chart of portfolio value over time)

This feature alone makes the project stand out from every other junior portfolio project.

3.3 Portfolio Tracker

User adds their real holdings: stock ticker, buy price, quantity, date. App shows current market value, profit/loss in rupees and percentage, and an AI suggestion (Hold or Sell) for each position based on current indicators.

3.4 Watchlist + Dashboard

User adds stocks to a watchlist. Dashboard shows all watchlist stocks with their current signal (Buy/Hold/Sell badge), latest price, and RSI value at a glance.

3.5 Daily Cron Alerts

APScheduler runs a job every morning at 9:00 AM IST. It loops through all users' watchlist stocks, runs the indicator calculations, and sends an email alert via Resend.com if any stock triggers a signal (RSI < 30, MACD bullish crossover, or price near 52-week low).

3.6 AI Suggestion Audit Log

Every AI suggestion is stored in the ai_log table with the date, ticker, signal, reasoning, and indicator values at the time. Users can view the history of suggestions for any stock and see retrospectively whether they were accurate. This is a mature engineering decision that makes the AI feature accountable.

---

4. Application Screens

Screen	Key Components	API Calls
Login / Signup	Email + password form, Supabase auth	POST /auth/signup, POST /auth/login
Dashboard	Watchlist table with signal badges, Add stock input	GET /watchlist, GET /stock/:ticker
Stock Detail	Price chart, RSI/MACD charts, AI suggestion card, Add to watchlist button	GET /stock/:ticker, GET /stock/:ticker/ai
Backtester	Stock picker, strategy selector, date range, results table + equity curve chart	GET /stock/:ticker/backtest
Portfolio	Holdings table with P&L, Add holding form, AI call per holding	GET /portfolio, POST /portfolio, GET /stock/:ticker/ai

---

5. API Endpoints

Method	Endpoint	Description
GET	/stock/:ticker	Fetch price history + RSI, MACD, MA calculations

GET	/stock/:ticker/ai	Run AI suggestion — calls LLM with indicator context
GET	/stock/:ticker/backtest	Run backtest — query params: strategy, from_date, to_date
GET	/watchlist	Get authenticated user's watchlist
POST	/watchlist	Add stock to watchlist
DELETE	/watchlist/:ticker	Remove stock from watchlist
GET	/portfolio	Get all holdings for authenticated user
POST	/portfolio	Add a new holding (ticker, buy_price, quantity, date)
DELETE	/portfolio/:id	Remove a holding
GET	/ai-log/:ticker	Get past AI suggestions for a specific stock

---

6. Database Schema (PostgreSQL / Supabase)

users

Managed by Supabase Auth. id (uuid PK), email, name, created_at.

stocks

ticker	VARCHAR PRIMARY KEY — e.g. TCS.NS, INFY.NS
name	VARCHAR — full company name
exchange	VARCHAR — NSE or BSE

watchlist

id	UUID PRIMARY KEY
user_id	UUID FK → users.id
ticker	VARCHAR FK → stocks.ticker
added_at	TIMESTAMP DEFAULT now()

portfolio

id	UUID PRIMARY KEY
user_id	UUID FK → users.id
ticker	VARCHAR FK → stocks.ticker
buy_price	DECIMAL(10,2) — price paid per share

quantity	INTEGER — number of shares
bought_on	DATE

ai_log

id	UUID PRIMARY KEY
user_id	UUID FK → users.id
ticker	VARCHAR FK → stocks.ticker
signal	VARCHAR — BUY, HOLD, or SELL
reasoning	TEXT — full LLM response
rsi_value	DECIMAL(5,2) — RSI at time of suggestion
macd_value	DECIMAL(10,4) — MACD at time of suggestion
created_at	TIMESTAMP DEFAULT now()

---

7. AI Advisor Prompt Template

This is the core prompt sent to the LLM for every stock analysis request. The backend fills in all the values before sending.

```
You are a stock research assistant for Indian retail investors.
Here is the current data for {ticker} ({company_name}):

Current price: ₹{price}
RSI (14-period): {rsi}
MACD: {macd}  |  Signal line: {macd_signal}
50-day MA: ₹{ma50}  |  200-day MA: ₹{ma200}
Volume trend: {volume_trend} (above/below 20-day average)

Historical pattern for this stock:
The last {n} times RSI dropped below 35, the average recovery
was {avg_recovery}% over {avg_days} days.

Based on this data, give a clear suggestion: BUY, HOLD, or SELL.
Explain your reasoning in 3-4 sentences. Be direct and specific.
Do not use financial advice disclaimers.
```

The response is stored in ai_log with the signal extracted (BUY/HOLD/SELL) and the full reasoning text.

---

8. Build Order

Week	Focus	What to build
1	Backend foundation	FastAPI setup, yfinance integration, /stock/:ticker endpoint returning price + RSI + MACD via pandas-ta
2	AI engine	Build the prompt, call Claude API, parse BUY/HOLD/SELL signal, save to ai_log table
3	Frontend	React + Vite setup, stock search, price chart (Recharts), indicator display, AI suggestion card
4	Auth + watchlist + portfolio	Supabase auth, watchlist endpoints + UI, portfolio tracker with P&L calculation
5	Backtesting engine	Hardest week — simulate trades over date range, return equity curve data, build results UI
6	Cron + email alerts	APScheduler morning job, Resend.com email, alert triggers (RSI < 30, MACD crossover)
7	Deploy + polish	Vercel (frontend) + Render (backend), environment variables, bug fixes, README

---

9. Project Folder Structure

Backend (Python / FastAPI)

```
backend/
  main.py                  # FastAPI app entry point
  requirements.txt
  .env                     # API keys, DB URL
  routers/
    stock.py               # /stock routes
    watchlist.py           # /watchlist routes
    portfolio.py           # /portfolio routes
    ai_log.py              # /ai-log routes
  services/
    stock_data.py          # yfinance fetch + indicator calc
    ai_advisor.py          # builds prompt + calls Claude API
    backtester.py          # backtesting engine logic
```

```
    pattern_engine.py      # historical pattern matching
  scheduler/
    cron.py                # APScheduler daily job
    alert_mailer.py        # Resend.com email sender
  db/
    database.py            # SQLAlchemy connection
    models.py              # ORM models
```

Frontend (React / Vite)

```
frontend/
  src/
    pages/
      Dashboard.jsx
      StockDetail.jsx
      Backtester.jsx
      Portfolio.jsx
      Login.jsx
    components/
      PriceChart.jsx
      IndicatorChart.jsx
      AISuggestionCard.jsx
      SignalBadge.jsx
      BacktestResults.jsx
    services/
      api.js               # all fetch calls to backend
      auth.js              # Supabase auth helpers
    App.jsx
    main.jsx
```

---

10. Environment Variables

Backend (.env)

```
ANTHROPIC_API_KEY=sk-ant-...
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_SERVICE_KEY=eyJ...
DATABASE_URL=postgresql://user:pass@host:5432/dbname
RESEND_API_KEY=re_...
FRONTEND_URL=https://your-app.vercel.app
```

Frontend (.env)

```
VITE_SUPABASE_URL=https://xxxx.supabase.co
VITE_SUPABASE_ANON_KEY=eyJ...
VITE_API_URL=https://your-backend.render.com
```

---

11. Resume Line (when done)

> *Built StockSense AI — a production-deployed stock research platform with LLM-powered analysis engine, historical pattern matching, backtesting (RSI/MACD strategies), daily cron-based watchlist alerts, and suggestion audit trail. Stack: Python, FastAPI, React, PostgreSQL, Claude API. Deployed on Vercel + Render.*

---

12. Important Notes for Development

- Never call this a 'prediction' app. Always frame it as an AI research assistant — suggestions based on data, not guaranteed outcomes.
- yfinance tickers for Indian stocks use .NS suffix for NSE (e.g. TCS.NS, RELIANCE.NS) and .BO for BSE.
- pandas-ta is the easiest way to compute RSI and MACD — install with: pip install pandas-ta
- For the Claude API call, always use model: claude-sonnet-4-20250514 and max_tokens: 1000
- Supabase Row Level Security (RLS) must be enabled — users should only see their own watchlist and portfolio rows.
- The backtesting engine is the hardest part — build it in Week 5 when you have the full context of the codebase.
- APScheduler runs inside the FastAPI process — no separate worker needed for Render free tier.
- All timestamps should be stored in UTC, converted to IST only in the frontend.